

Delivery Route Optimization System Using Graph Algorithms

PROJECT DOCUMENTATION

1. Project Overview

Student Profiles

- **Student Name:** T. Vinitha (Roll No: 23AK1A3256)
- **Student Name:** S. Harsha Priya (Roll No: 23AK1A3214)
- **Department:** B. Tech - Computer Science and Engineering (Data Science)
- **College Name:** Annamacharya Institute of Technology and Sciences, Tirupati
- **Academic Year:** 2025-2026

Project Description

The **Delivery Route Optimization System** is a software application designed to solve complex logistics and supply chain distribution challenges using mathematical graph theory. It models physical distribution hubs and delivery locations as vertices (nodes) and interconnecting roads or transit lines as weighted edges (representing distances or travel times). By executing graph search algorithms, the system computes the absolute shortest paths across a network to minimize fuel consumption, lower operational costs, and reduce shipping delays.

Core Objectives

- **Minimize Transit Overhead:** Determine the most efficient routes between multi-point delivery nodes.
- **Optimize Logistics Operations:** Apply computing structures to streamline real-world fleet management.
- **Reduce Operational Time:** Automatically bypass inefficient paths using optimal weight-discovery procedures.
- **Apply Graph Theory:** Implement foundational algorithmic concepts to solve physical shipping bottlenecks.

2. Technical Stack

- **Programming Language:** Python 3.12 (Base environment via Anaconda)
- **Algorithm Base:** Dijkstra's Shortest Path Algorithm
- **IDE/Development Environment:** Visual Studio Code (VS Code)
- **Version Control:** Git & GitHub Repository Management

3. Problem Statement

Modern distribution and courier services handle expanding networks of fulfillment points. Selecting paths manually or relying on basic sequencing often results in redundant paths, directly increasing vehicle wear, driver travel time, and operational fuel expenses. The proposed application builds a dynamic computation pipeline that analyzes arbitrary location clusters and identifies the lowest-cost path, establishing highly efficient transit configurations.

4. System Requirements

Hardware Requirements

- **Machine:** Modern Desktop or Laptop (Windows/macOS/Linux)
- **Connectivity:** Internet access for version control syncing and package installations

Software Requirements

- **Runtime Environment:** Python 3.x interpreter
- **Development Platform:** Visual Studio Code with Python Extensions configured
- **Repository Architecture:** GitHub Platform Account and Git Core Tools

5. Algorithm Architecture & Data Structures

The system represents the structural delivery map using an **adjacency list** formatted inside a Python dictionary. Every key represents an origin node, and its associated dictionary maps target nodes alongside their precise edge costs.

Network Graph Data Structure

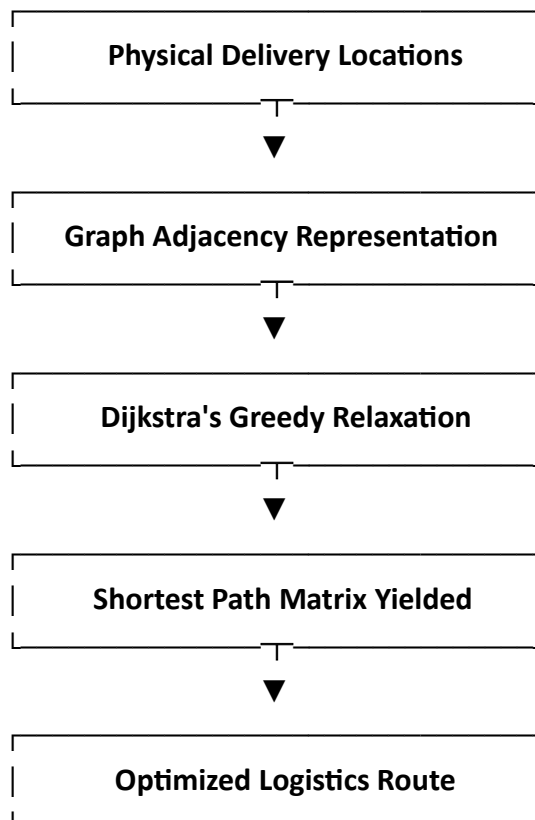
Python

```
graph = {  
    'A': {'B': 4, 'C': 2},  
    'B': {'A': 4, 'C': 1, 'D': 5},  
    'C': {'A': 2, 'B': 1, 'D': 8, 'E': 10},  
    'D': {'B': 5, 'C': 8, 'E': 2, 'F': 6},  
    'E': {'C': 10, 'D': 2, 'F': 3},  
    'F': {'D': 6, 'E': 3}  
}
```

Core Algorithmic Steps (Dijkstra's Approach)

1. **Distance Tracking Initialization:** Define the distance from the source node to itself as 0, and set all other unvisited node paths to infinity (∞).
2. **Greedy Node Selection:** Evaluate the pool of unvisited nodes and select the node with the lowest calculated distance metric.
3. **Neighbor Evaluation & Edge Relaxation:** Examine all neighboring nodes connected to the current node. Calculate their tentative distance values through the active node. If this new path cost is lower than the previously recorded cost, update the distance record.
4. **Visited Mark Allocation:** Push the evaluated node into the visited array to prevent duplicate computation cycles.
5. **Termination Loop:** Re-run steps 2 through 4 until every node inside the network framework has been processed.

6. System Design



7. Complete Production Code

Source File (main.py)

Python

```
# Delivery Route Optimization System using Dijkstra's Algorithm
```

```
# Defining the delivery network configuration using an Adjacency List
```

```
graph = {  
    'A': {'B': 4, 'C': 2},  
    'B': {'A': 4, 'C': 1, 'D': 5},  
    'C': {'A': 2, 'B': 1, 'D': 8, 'E': 10},  
    'D': {'B': 5, 'C': 8, 'E': 2, 'F': 6},  
    'E': {'C': 10, 'D': 2, 'F': 3},  
    'F': {'D': 6, 'E': 3}  
}
```

```
def dijkstra(graph, start):
```

```
    # Step 1: Initialize all distances as infinite, and the starting node as 0
```

```
    distances = {node: float('inf') for node in graph}
```

```
    distances[start] = 0
```

```
    visited = []
```

```
    # Step 2: Loop until all nodes are fully processed
```

```
    while len(visited) < len(graph):
```

```
        # Select the unvisited node with the smallest calculated distance
```

```
        current = min(  
            (node for node in graph if node not in visited),  
            key=lambda node: distances[node]  
        )
```

```
        visited.append(current)
```

```
    # Step 3: Evaluate weights and relax neighboring paths
```

```
    for neighbor, weight in graph[current].items():
```

```
        distance = distances[current] + weight
```

```
        if distance < distances[neighbor]:
```

```
            distances[neighbor] = distance
```

```
return distances
```

```
# Program Execution Entrypoint

if __name__ == "__main__":
    start_node = 'A'

    result = dijkstra(graph, start_node)

    print("Delivery Route Optimization System")

    print(f"Shortest distances from {start_node}")

    for node, distance in result.items():
        print(f"{node}: {distance}")
```

8. Execution Results & Functional Validation

When execution initiates on the underlying architecture using node 'A' as the centralized distribution hub, the program resolves the following optimal metrics:

Output Verification Matrix

Destination Node	Shortest Path Weight from Hub 'A'	Optimized Path Sequence
A	0	Origin Hub
B	3	A → C → B
C	2	A → C
D	8	A → C → B → D
E	10	A → C → B → D → E
F	13	A → C → B → D → E → F

Optimization Proof Note: A direct connection path is configured from $A \rightarrow B$ with a hardcoded edge cost value of **4**. However, the system bypasses this route, discovering a more efficient path through Node C ($A \rightarrow C \rightarrow B$), yielding a lower overall total weight of **3**. This validates that the greedy relaxation phase functions correctly.

9. Repository Structure Tracking

The project components as verified on your production GitHub repository (vinitha-09/Delivery-Route-Optimization-System) use the following file mapping layout:

Plaintext

Delivery-Route-Optimization-System/

|

|— Screenshots/

| |— Code_Screenshot1.png

| |— Code_Screenshot2.png

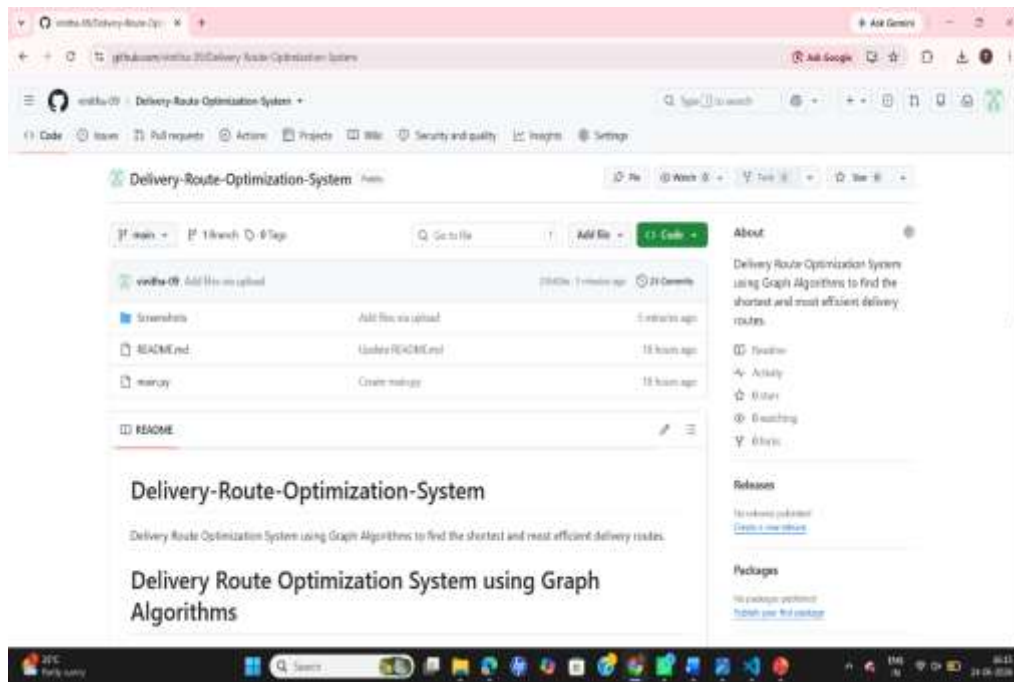
| |— Program_Output.png

|

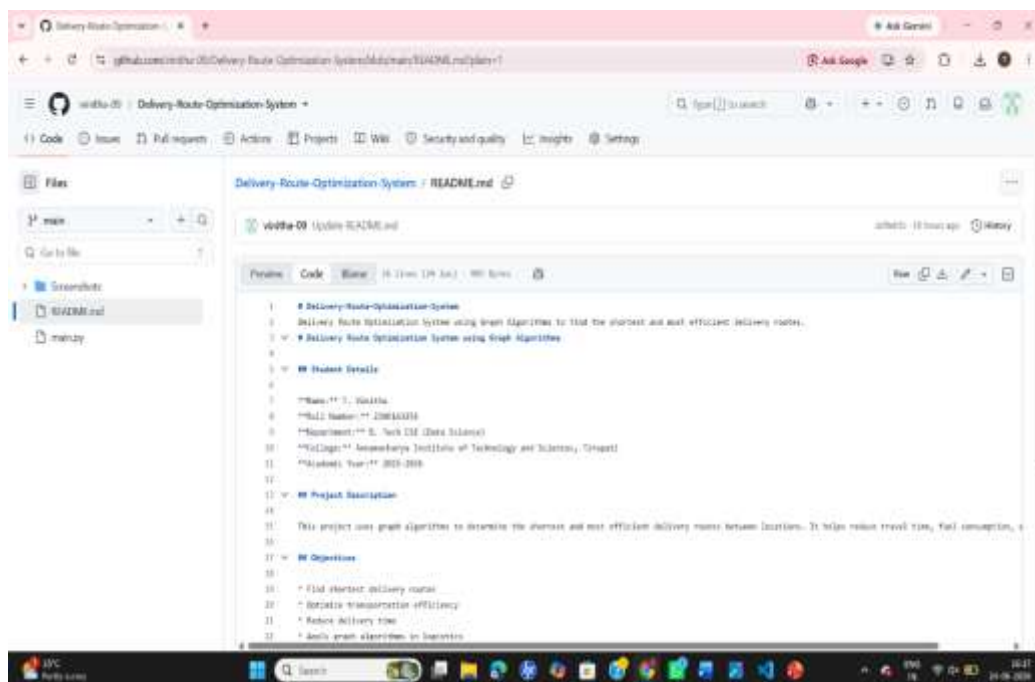
|— README.md

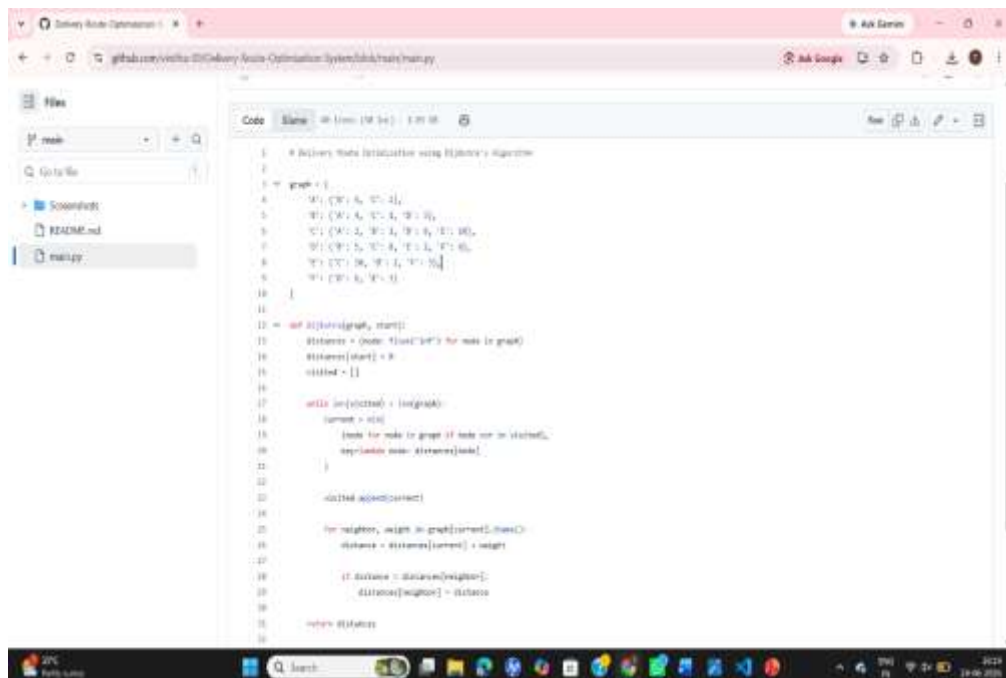
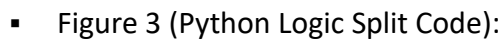
|— main.py

- main.py: Holds the production source script executing the routing solution.
- README.md: Hosts academic headers, abstract goals, objectives list, and platform stack components.
- Screenshots/: Directory containing production execution captures and terminal validation outputs.
 - *File References:*
 - Figure 1 (Repository Home Page):



▪ Figure 2 (README Markup Plain View):





The screenshot shows a web browser window displaying a GitHub repository titled "Delivery Route Optimization System". The file explorer on the left shows files named "main.py", "README.md", and "env.py". The main.py file is open in a code editor, showing the following Python code:

```

11 def dijkstra(graph, start):
12     distances = {node: float('inf') for node in graph}
13     distances[start] = 0
14     visited = []
15
16     while len(visited) < len(graph):
17         current = min(
18             [node for node in graph if node not in visited],
19             key=lambda node: distances[node]
20         )
21         visited.append(current)
22
23         for neighbor, weight in graph[current].items():
24             distance = distances[current] + weight
25
26             if distance < distances[neighbor]:
27                 distances[neighbor] = distance
28
29         return distances
30
31 start_node = 'A'
32 result = dijkstra(graph, start_node)
33
34 print('Delivery Route Optimization System')
35 print('Shortest distances from', start_node)
36
37 for node, distance in result.items():
38     print(f'{node}: {distance}')
  
```

Figure 4 (Output):

The screenshot shows a Jupyter Notebook environment with the same Python code as in Figure 4. The code is executed, and the output is displayed in the console. The output shows the shortest distances from node 'A' to all other nodes in the graph:

```

Delivery Route Optimization System
Shortest distances from A
A: 0
B: 3
C: 2
D: 6
E: 10
F: 11
  
```

10. Advantages & System Applications

Advantages

- **Optimized Efficiency:** Guarantees computation of the single-source shortest path across arbitrary non-negative graphs.
- **Resource Preservation:** Bypasses manual logistics tracking, shrinking planning phases from hours down to milliseconds.
- **Scalability:** Easily integrates within modern object-oriented code modules to represent real-time changes in weights (like traffic or route closures).

Applications

- Commercial e-commerce and home supply drop operations.
- Enterprise supply-chain network configuration.
- On-demand mobile food transportation structures.
- Intelligent routing protocols inside local telecommunication setups.

11. Conclusion

The **Delivery Route Optimization System Using Graph Algorithms** provides a robust implementation showing how discrete structures can solve physical fleet optimization issues. By integrating Dijkstra's algorithm with clean data mapping formats, the development framework successfully processes destination hubs and outputs optimal target weights. The execution outputs verified inside terminal captures prove the technical viability of this tool for integration into wider corporate supply architectures.

12. References

- **Python Standard Library Documentation:** Core structural mapping configurations.
- **GitHub User Manuals:** Production branch hosting and version controls.
- **Dijkstra, E. W. (1959):** *A note on two problems in connexion with graphs*. Numerische Mathematik.
- **Data Structures and Algorithms Core Textbooks:** Greedy choices and relaxation mechanics.